Term Project

ECE 509 (Spring 2026)

Course number: 16:332:509

Instructor: Waheed U. Bajwa (waheed.bajwa@rutgers.edu)

1 Project Overview

The term project is a substantial independent study in optimization, with an emphasis on technical understanding, numerical implementation, and professional communication.

Students may work on any topic that is meaningfully connected to optimization, including convex optimization and adjacent areas. A curated list of sample topics appears later in this document in Section 9. That list is not exhaustive; topics outside the list are welcome, but they should be discussed with the instructor early enough to avoid scope or alignment issues.

The project should look and feel like a small research or technical development effort: identify a topic, study the relevant literature, implement meaningful numerical experiments, interpret the results, write a technical report, and present the work orally.

2 Learning Objectives

The project is intended to help students:

- develop and demonstrate genuine understanding of optimization concepts beyond what was covered directly in class;
- implement and evaluate optimization methods or optimization-based models numerically;
- communicate technical ideas clearly in professional written form;
- present technical material orally and answer technical questions about the work.

3 Team Policy

The default expectation is a team of 2 students. A team of 3 students may be approved if the proposal makes a convincing case that the project scope is broad enough to justify a third member. Individual projects are also allowed, but the scope should still be substantial and realistic for one person.

Each team must identify one point of contact (POC). The POC is responsible for submitting the team's materials on Canvas and for ensuring that the Canvas submission clearly identifies all team members.

Each team is jointly responsible for the quality and integrity of the final submission. Team members may divide labor, but all members are expected to understand the problem, methods, code structure, experiments, and conclusions. During the presentation and Q&A, any team member may be asked about any part of the project.

3.1 Team Contribution and Nonparticipation

Every listed team member is expected to have materially contributed to the project and to be able to discuss the work during the presentation and Q&A.

If a teammate stops responding or materially stops contributing, the remaining student(s) must contact the instructor promptly. Be prepared to document sincere, good-faith efforts to reach out and resolve the issue. After review, expectations, scope, team structure, or grading treatment may be adjusted as appropriate. Do not wait until the final deadline to report a serious team problem.

4 Project Phases and Deliverables

Phase	Weight	Deadline / Schedule
Phase I – Project Initiation / Proposal	10%	April 8, 2026 at 11:59 PM
Phase II – Report Development	60%	May 4, 2026 at 11:59 PM
Phase III – Final Presentation	30%	May 6, 2026

4.1 Phase I – Project Initiation / Proposal (10%)

Submit one PDF per team on Canvas. The submission must be made by the team's POC, and the Canvas submission must clearly identify the other team members. The proposal narrative should be no more than 500 words, excluding references and the list of team members. It should include:

- team members and a brief description of intended roles or task split;
- project topic, motivation, and why the topic is worth studying;
- intended scope, including the optimization content the project will cover;
- an initial technical plan, including what the team expects to implement or evaluate numerically;
- at least three relevant papers or comparable technical references that will anchor the project initially;
- concise success criteria for the project;
- if requesting a third teammate, a specific justification for why two students would not be enough for the proposed scope.

The proposal should be specific enough for the instructor to judge scope, technical direction, and feasibility. Broad topic declarations without a clear plan are not sufficient.

4.2 Phase II – Report Development (60%)

For Phase II – Report Development, submit one report PDF on Canvas. That PDF must include the GitHub repository link; see Section 5.3. If LLMs or similar tools were used, the required disclosure materials must be kept in the repository as described in Section 6.2.

The submission must be made by the team's POC, and the Canvas submission must clearly identify the other team members.

The report must:

- be written in a professional technical style;

- use third person throughout;
- contain an abstract, introduction, technical background, main method or content section, numerical results section, conclusion, and bibliography;
- contain approximately 2,000 to 4,000 words;
- clearly distinguish background material from the students' own analysis, implementation, experiments, and interpretation;
- include enough detail for the numerical results to be evaluated seriously.

IEEE style is recommended for the bibliography.

LaTeX is recommended for the written report. Overleaf is a reasonable option for teams not already comfortable with a local LaTeX workflow. LaTeX is not required.

For the report word count, references, appendices, and figure/table captions do **not** count. LLM disclosure materials kept in the repository, such as `llm-usage.md`, also do **not** count toward the report word limit.

4.3 Phase III – Final Presentation (30%)

Each team will give a 15 minutes presentation followed by a 5 minutes question-and-answer period. Sign-up details will be provided later.

Slides must also be uploaded to Canvas as a PDF. That upload must be made by the team's POC, and the Canvas submission must clearly identify the other team members.

The presentation should summarize the technical problem, the methods studied, the main implementation details, the key numerical results, and the principal takeaways. The presentation will be evaluated on clarity, technical quality, slide quality, and the ability to answer questions accurately and directly. The presentation and Q&A are also used to assess the team's understanding and ownership of the written report. If the presentation reveals a serious mismatch between the report and the team's demonstrated understanding, that may affect the report grade as well.

5 Submission and Reproducibility Requirements

Canvas is the official submission point for all phases. Only PDFs should be uploaded there. For team submissions, the POC submits on Canvas and must clearly identify the other team members. For Phase II – Report Development, the report PDF must include the GitHub repository link, and the repository must contain any required LLM disclosure and supporting material.

5.1 Submission Map

Phase	Required artifacts	Submitted where	Accompanying links / notes
Phase I – Project Initiation / Proposal	Proposal PDF	Canvas	One PDF per team; submitted by POC; list all team members
Phase II – Report Development	Report PDF	Canvas	Submitted by POC; list all team members; include repository link in the PDF; repository contains LLM materials if applicable
Phase III – Final Presentation	Slides PDF; live presentation	Canvas / in class	Presentation delivered live; slides PDF uploaded on Canvas by POC; list all team members

5.2 File and Naming Expectations

- Submit one proposal PDF per team for Phase I – Project Initiation / Proposal.
- Submit one final report PDF per team for Phase II – Report Development.
- Submit one slides PDF per team for Phase III – Final Presentation.
- Every team Canvas submission should be made by the POC and should list the other team members.
- Use a simple filename based on the project phase, the POC's last name, and the initials of all team members so filenames remain distinct even when POC last names coincide. For example: `phase1-bajwa-wb-jq.pdf`, `phase2-bajwa-wb-jq.pdf`, or `phase3slides-bajwa-wb-jq.pdf`.

5.3 Repository Expectations

The GitHub repository for Phase II – Report Development should be linked clearly in the submitted report PDF and should include:

- source code used for the reported numerical results;
- a short `README` explaining how to run the main experiments or reproduce the reported outputs;
- a clear list of required packages, libraries, environments, or data dependencies;
- any scripts, notebooks, or configuration files needed to understand the reported experiments;
- clear identification of external code or adapted code, if any;
- the LLM disclosure and supporting material required by Section 6.2, if applicable.

The repository should be organized so the instructor can inspect and follow the reported work. If results depend on data, parameters, or random seeds, document that clearly. The repository must be in its final submitted state by the Phase II – Report Development deadline. Do not modify it after the deadline unless the instructor explicitly permits that change.

6 LLM and External-Tool Usage Policy

Large language models and related AI tools may be used, but all such use must be disclosed clearly enough for the instructor to review.

6.1 Required Disclosure

If any LLM or similar tool is used, the team must disclose:

- the tool or platform used;
- the provider and model name, if known;
- the dates on which the tool was used;
- what parts of the project it was used for;
- the prompts given;
- the outputs that were materially relied upon.

6.2 Required Sharing and Logging

If the platform supports sharing a project, conversation, or workspace, the team must share it for review with `waheed.bajwa@rutgers.edu` and record that shared link in the repository disclosure artifact. If the platform does not support sharing, the team must store exported conversation logs, screenshots, or a prompt transcript in the repository.

Each team must also keep a dedicated disclosure artifact in the repository, such as `llm-usage.md`. That artifact must include:

- the tool/model used;
- date(s) of use;
- a prompt log or transcript;
- a summary of how the outputs were checked, edited, or rewritten;
- a statement confirming that the students take full responsibility for the final work.

6.3 Responsibility and Restrictions

The following rules apply:

- Students remain fully responsible for correctness, writing quality, code quality, citations, and all technical claims.
- Undisclosed LLM use is a course-integrity issue.
- LLM use does not replace understanding; oral questioning may probe any part of the report, code, slides, or experiments, and a polished report is not enough if the team cannot explain the work.
- Disclosure does not excuse lack of understanding. If LLM-generated or LLM-assisted material is used and the team cannot explain it, that is also an integrity issue.
- Any generated material that is used must be reviewed, verified, edited, and technically owned by the students.
- The report, repository, slides, references, and disclosed tool usage must be consistent with one another.
- Do not upload confidential or private data, and do not upload material in ways that violate platform terms or copyright restrictions.

- Do not present unverified LLM-generated text, code, citations, proofs, or results as if they were reliable by default.
- If LLM assistance materially shaped exposition, code, experiments, or technical framing, that assistance must be documented explicitly.

7 Evaluation Criteria

The project will be judged holistically, but the following criteria will matter throughout:

- **Understanding and ownership:** depth of understanding of the chosen topic, ability to explain the work clearly, and demonstrated ownership during the presentation and Q&A.
- **Scope and technical quality:** clarity of topic, appropriateness of scope, correctness of the technical content, and seriousness of the optimization work.
- **Implementation and results:** quality of implementation, relevance of experiments, and quality of interpretation.
- **Communication:** quality of the report, slides, presentation, and responses during Q&A.
- **Integrity and documentation:** proper citation, clear disclosure of assistance and tools, and reproducibility of reported work.

7.1 Grade Expectations

As a rough guide, work that solidly meets the expected scope and quality of the project will be around 80%. To score meaningfully above that, a team must do substantially more in technical depth, execution, reproducibility, communication, and demonstrated understanding during the presentation and Q&A.

8 Administrative Notes

- If you want to pursue a topic outside the suggested list, discuss it with the instructor early.
- If team formation is difficult, use the course-provided teammate-search mechanism early rather than waiting until the proposal deadline.
- Scope matters. A narrow project done carefully is better than a broad project done shallowly.
- The report, repository, and presentation should reflect what the team actually understands and implemented.
- Identify copied or adapted code clearly in the repository, and cite only references you actually used.

9 Suggested Topic Areas

The list below is curated, not exhaustive. These topics are examples of acceptable directions rather than a closed menu.

1. **Stochastic Gradient Descent (SGD):** Study stochastic updates for large-scale objectives. Applications include neural-network training and recommendation systems.

2. **Accelerated Gradient Descent:** Study methods that improve convergence relative to basic gradient descent. Applications include large-scale machine learning and data analysis.
3. **Incremental Gradient and Coordinate Descent:** Study update rules that use partial information at each step. Applications include online learning, sparse optimization, and feature selection.
4. **Mirror Descent:** Study first-order optimization with problem-adapted geometry. Applications include online optimization, economics, and reinforcement learning.
5. **Quasi-Newton Methods:** Study Hessian-approximation methods that aim for faster convergence without full second derivatives. Applications include nonlinear parameter estimation and large-scale optimization.
6. **Conjugate Gradient Methods:** Study methods for linear systems and unconstrained optimization. Applications include numerical linear algebra, regression, and computational physics.
7. **Trust-Region Methods:** Study methods that optimize a local model within a controlled neighborhood. Applications include nonlinear parameter estimation and engineering design.
8. **Augmented Lagrangian and ADMM:** Study decomposition-oriented methods for constrained optimization. Applications include distributed computing, consensus optimization, and machine learning.
9. **Distributed Optimization:** Study optimization over networks or decentralized systems. Applications include resource allocation, decentralized learning, and smart grids.
10. **Optimization on Manifolds:** Study optimization over structured geometric spaces. Applications include robotics, computer vision, and low-dimensional representation methods.
11. **Proximal Optimization:** Study algorithms based on proximal operators for nonsmooth problems. Applications include compressed sensing, image reconstruction, and signal processing.
12. **Subgradient Methods:** Study methods for nondifferentiable convex objectives. Applications include support vector machines and general convex programming.
13. **Robust Optimization:** Study decision-making under uncertainty in optimization models. Applications include supply chains, finance, and engineering design.
14. **Online Optimization:** Study optimization with sequentially arriving data. Applications include dynamic pricing, advertising, and real-time scheduling.
15. **Derivative-Free Optimization:** Study optimization without direct gradient information. Applications include simulation-based design, black-box tuning, and hyperparameter search.
16. **Bi-level Optimization:** Study hierarchical optimization where one problem depends on another problem's solution. Applications include hyperparameter tuning, economics, and resource allocation.
17. **Min-max / Saddle-Point Problems:** Study optimization problems with adversarial or game-theoretic structure. Applications include robust control, adversarial learning, and game theory.
18. **Multi-objective Optimization:** Study methods that balance multiple competing objectives. Applications include engineering design, economics, environmental planning, and healthcare.